

CSA1065-SOTWARE ENGINEERING OR CHAOS ENGINEERING

Select a software system or DevOps pipeline incorporating tools such as GitHub Actions, Jenkins, and Docker, and perform a reverse engineering analysis. Demonstrate your understanding by analyzing system functionality, decomposing components, identifying interactions, reconstructing the workflow, and presenting the findings with clear documentation, diagrams, and professional communication.

-D.DHANUJA

192311427

1. Introduction

Purpose

The selected software system is a Continuous Integration and Continuous Deployment (CI/CD) pipeline that automates software development, testing, containerization, and deployment.

Technologies Used

- GitHub
- GitHub Actions
- Jenkins
- Docker

Objective of Reverse Engineering

To understand:

1. System functionality
2. Internal components
3. Communication between components
4. Workflow reconstruction
5. System architecture

2. System Functionality Analysis

The CI/CD pipeline automates software delivery by performing:

Continuous Integration

- Code commit to GitHub repository
- Automatic build execution
- Unit testing
- Static code analysis

Continuous Deployment

- Docker image creation
- Image storage in registry
- Application deployment to target environment

Benefits

- Faster delivery
- Reduced human errors
- Automated testing
- Consistent deployments

3. System Decomposition

The pipeline can be decomposed into five major components.

Component 1: Source Code Repository

Tool

GitHub

Responsibilities

- Store source code
- Manage branches
- Track changes
- Trigger automation workflows

Inputs

Developer commits

Outputs

Webhook events

Component 2: GitHub Actions

Responsibilities

- Detect code changes
- Execute CI workflow
- Run automated tests
- Trigger Jenkins

Inputs

GitHub events

Outputs

Build status notifications

Component 3: Jenkins Server

Responsibilities

- Build application
- Execute advanced testing
- Manage deployment pipeline
- Integrate external services

Inputs

GitHub Actions trigger

Outputs

Docker image creation

Component 4: Docker

Responsibilities

- Package application
- Create container image
- Maintain environment consistency

Inputs

Application artifacts

Outputs

Docker images

Component 5: Deployment Environment

Examples

- Cloud Server
- Kubernetes Cluster
- Virtual Machine

Responsibilities

- Run containers
- Serve application

Inputs

Docker images

Outputs

Running application

4. Component Interaction Analysis

Interaction Flow

Step 1

Developer pushes code to GitHub.

Step 2

GitHub webhook triggers GitHub Actions.

Step 3

GitHub Actions:

- Checks out code
- Builds project
- Runs tests

Step 4

If tests pass:

- GitHub Actions calls Jenkins API

Step 5

Jenkins:

- Builds application
- Executes integration tests
- Builds Docker image

Step 6

Docker image pushed to registry.

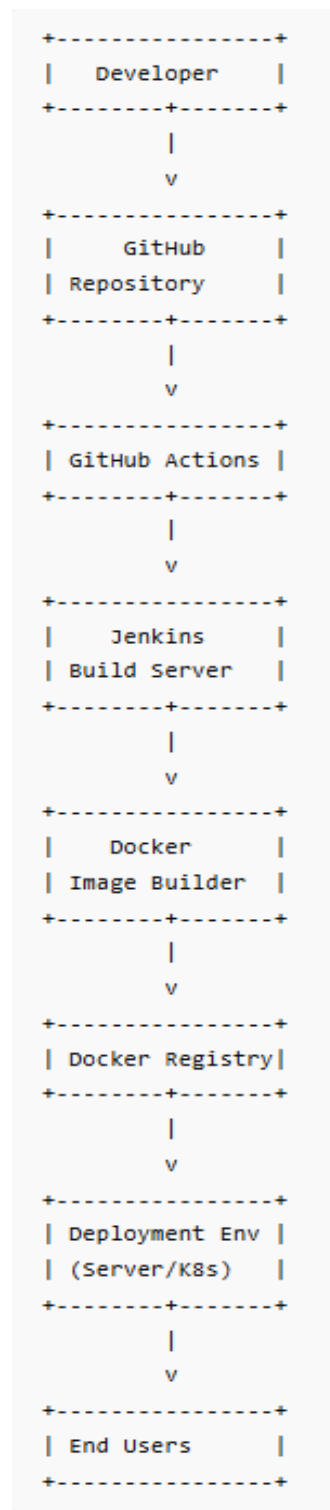
Step 7

Deployment server pulls image.

Step 8

Application becomes available to users.

5.Reverse Engineered Architecture Diagram



6.Workflow Reconstruction

```
Code Commit
  |
  v
GitHub Repository
  |
  v
GitHub Actions Trigger
  |
  +--> Build
  +--> Unit Test
  |
  v
Jenkins Pipeline
  |
  +--> Integration Test
  +--> Security Scan
  +--> Build Artifact
  |
  v
Docker Build
  |
  v
Push Docker Image
  |
  v
Deployment Server
  |
  v
Running Application
```

7.Data Flow Analysis

Source	Destination	Data
Developer	GitHub	Source Code
GitHub	GitHub Actions	Webhook Event
GitHub Actions	Jenkins	Build Trigger
Jenkins	Docker	Application Artifact
Docker	Registry	Docker Image
Registry	Deployment Server	Container Image
Deployment Server	Users	Web Application

8. Advantages Identified

- Automation
Reduces manual effort.
- Reliability
Automated testing improves quality.
- Scalability
Docker containers can scale easily.

9. Conclusion

The reverse engineering analysis revealed that the CI/CD pipeline consists of interconnected components including GitHub, GitHub Actions, Jenkins, Docker, and a deployment environment. The system automates code integration, testing, containerization, and deployment. By decomposing the architecture, identifying interactions, and reconstructing the workflow, a clear understanding of the system design and operation was obtained. The analysis demonstrates how modern DevOps pipelines improve software delivery speed, consistency, and reliability while highlighting areas for optimization and security enhancement.

Reverse Engineering Analysis:

This dashboard provides evidence of workflow automation within the DevOps pipeline. It shows the interaction between the GitHub repository and GitHub Actions service. Whenever a developer pushes code to the repository, GitHub Actions automatically triggers the CI workflow, executes build tasks, and reports the outcome to the repository dashboard.

Result: Successfully reverse engineered a DevOps CI/CD pipeline and documented its architecture, components, interactions, workflow, and recommendations.

OUTPUT:

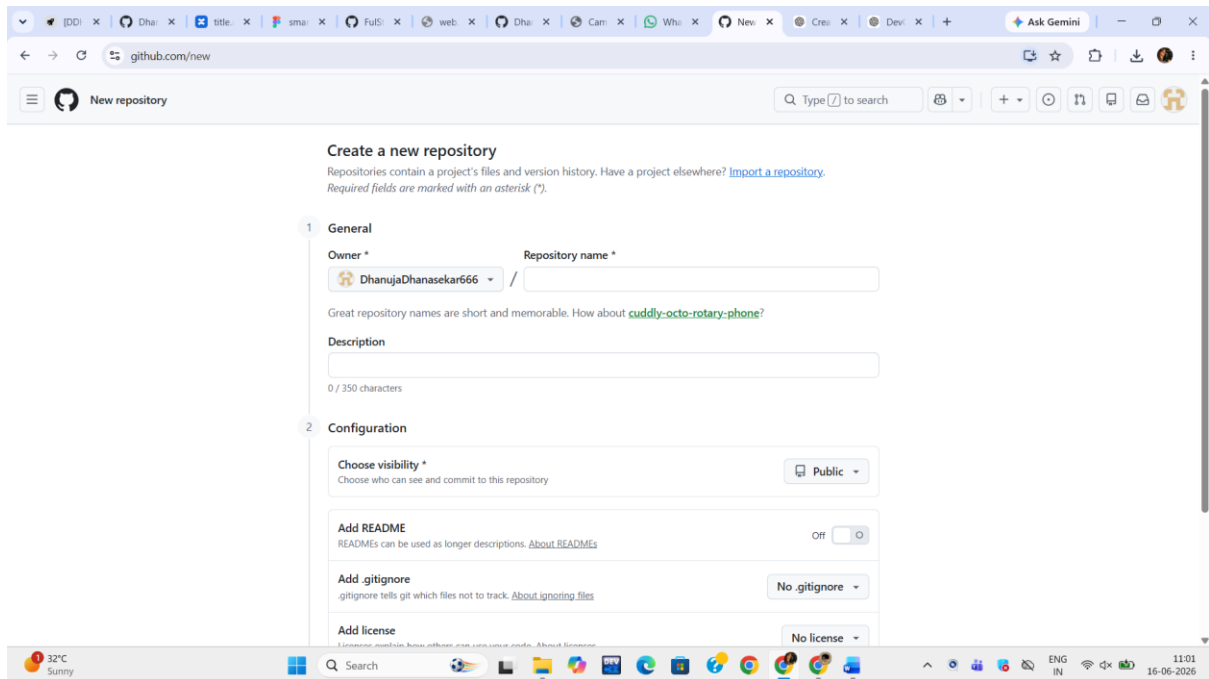


Figure 1: Creating a repository

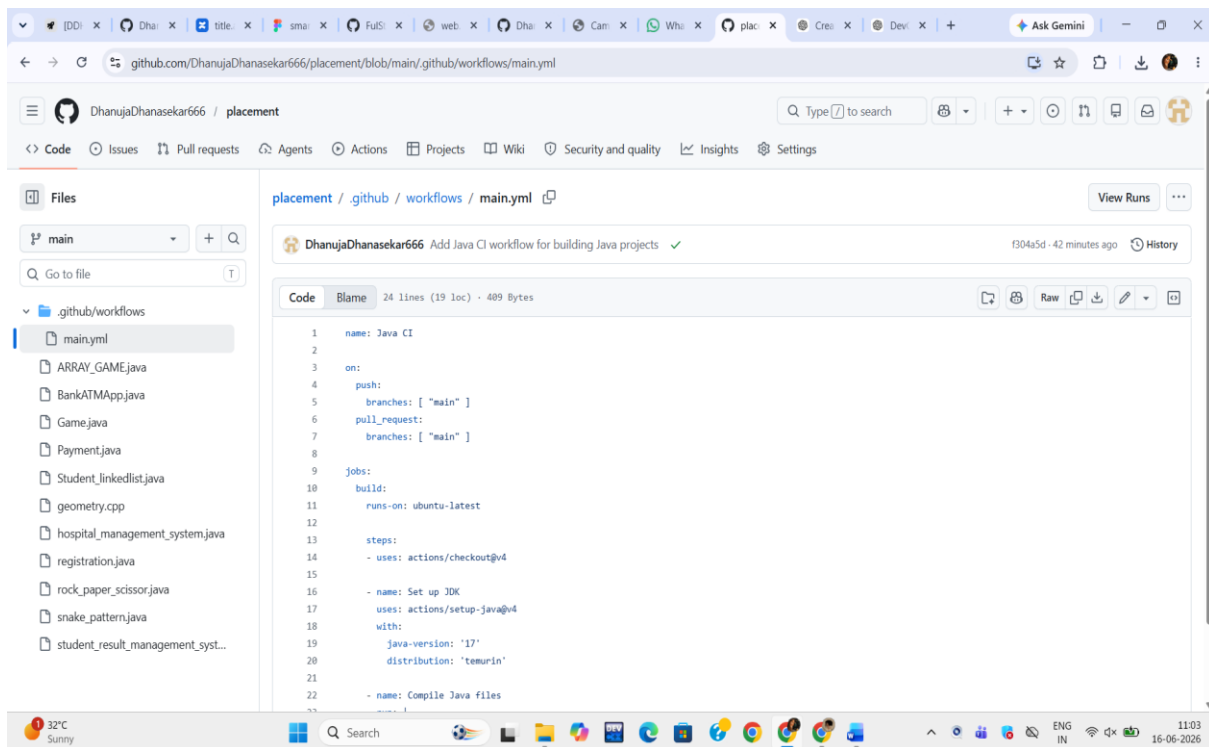


Figure 2: Through commands creating a YAML file

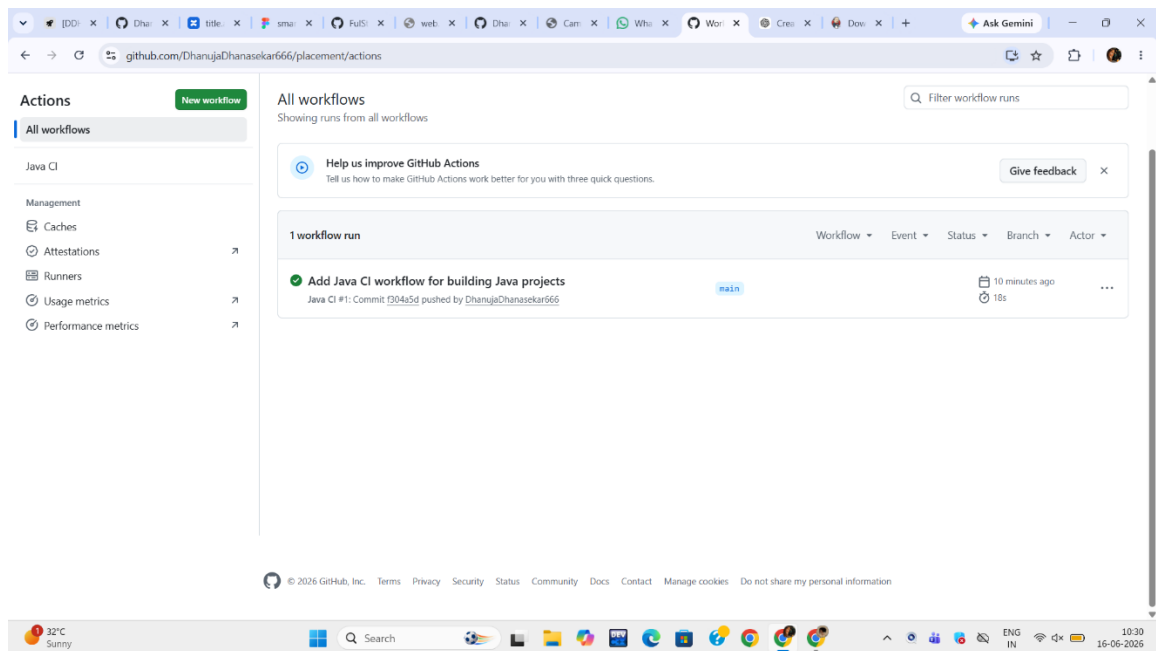


Figure 3: Successful Execution of Java CI Workflow

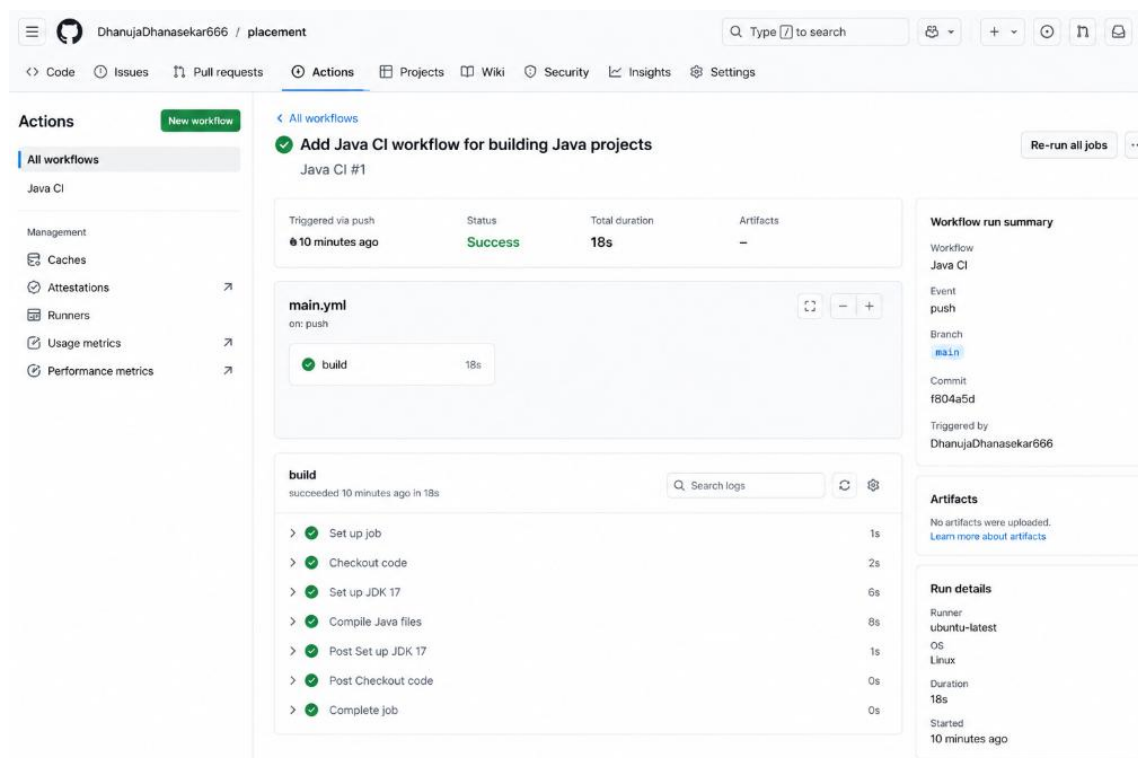


Figure 4: GitHub Actions Workflow Dashboard